Grace Brown

CSC 1310

Professor Crockett

27 October 2026

Program Two Part Two

Sample Output ONE:

```
hannah_grace@GraceLaptop:/mnt/c/Users/hanna/OneDrive/Documents/GitHub/School-Projects/CSC 1310/Programs/Program 2/Part 1$ ./prog2

How many threads do you want OpenMP to use? (Enter 1-500) 4

Number of Threads Used: 4
Number of Processors: 8

What is the name of the string (word) text file?
(wordsSmall.txt or wordsLarge.txt)
FILENAME: wordsLarge.txt

What is the name of the number (int) text file?
(numsSmall.txt or numsLarge.txt)
FILENAME: numsLarge.txt
SIZE of words: 31100
SIZE of nums: 37084

<parallel/algorithm> library sort(int): 0.00664971 seconds
<parallel/algorithm> library sort(words): 0.0103731 seconds

Bubble Sort (int): 10.3488 seconds

Bubble Sort Runtime (String): 22.1461 seconds

Merge Sort (int): 0.0352025 seconds

Merge Sort Runtime (String): 0.0676008 seconds

Parallel Merge Sort (int): 0.0252479 seconds

Parallel Merge Sort Runtime (String): 0.0531601 seconds

Quick Sort (int): 0.0127603 seconds

Quick Sort Runtime (String): 0.0212944 seconds

Parallel Quick Sort (int): 0.00955853 seconds

Parallel Quick Sort Runtime (String): 0.0266991 seconds
hannah_grace@GraceLaptop:/mnt/c/Users/hanna/OneDrive/Documents/GitHub/School-Projects/CSC 1310/Programs/Program 2/Part 1$
```

a. Why do you think you got the results that you got?

I believe this was a result of the computer being able to divide work more evenly through the extra threads, leaving the parallel sorts at a quicker time than, say, the bubble sorts.

b. What algorithm was the fastest out of the algorithms you implemented?

The fasted algorithm was Parallel Quick Sort

c. What algorithm was the slowest?

The slowest was Bubble Sort

d. How does your results compare to the sample output provided?

Although my numbers are different, the efficiency of each sort algorithm is the same with Bubble being the slowest.

Sample Output TWO:

```
hannah_grace@GraceLaptop:/mnt/c/Users/hanna/OneDrive/Documents/GitHub/School-Projects/CSC 1310/Programs/Program 2/Part 1$ ./prog2

How many threads do you want OpenMP to use? (Enter 1-500) twenty
Please enter a number 1-500: duh
Please enter a number 1-500: -489
Please enter a number 1-500: 20

Number of Threads Used: 20
Number of Processors: 8

What is the name of the string (word) text file?
(wordsSmall.txt or wordsLarge.txt)
FILENAME: wordsLarge.txt

What is the name of the number (int) text file?
(numsSmall.txt or numsLarge.txt)
FILENAME: numsLarge.txt
SIZE of words: 31100
SIZE of nums: 37084

<parallel/algorithm> library sort(int): 0.0186722 seconds
<parallel/algorithm> library sort(words): 0.0169889 seconds

Bubble Sort (int): 10.3793 seconds

Bubble Sort Runtime (String): 21.1474 seconds

Merge Sort (int): 0.0406047 seconds

Merge Sort Runtime (String): 0.0727056 seconds

Parallel Merge Sort (int): 0.0197432 seconds

Parallel Merge Sort Runtime (String): 0.0545764 seconds

Quick Sort (int): 0.0098878 seconds

Quick Sort Runtime (String): 0.0248129 seconds

Parallel Quick Sort (int): 0.00928536 seconds

Parallel Quick Sort Runtime (String): 0.0320808 seconds
hannah_grace@GraceLaptop:/mnt/c/Users/hanna/OneDrive/Documents/GitHub/School-Projects/CSC 1310/Programs/Program 2/Part 1$
```

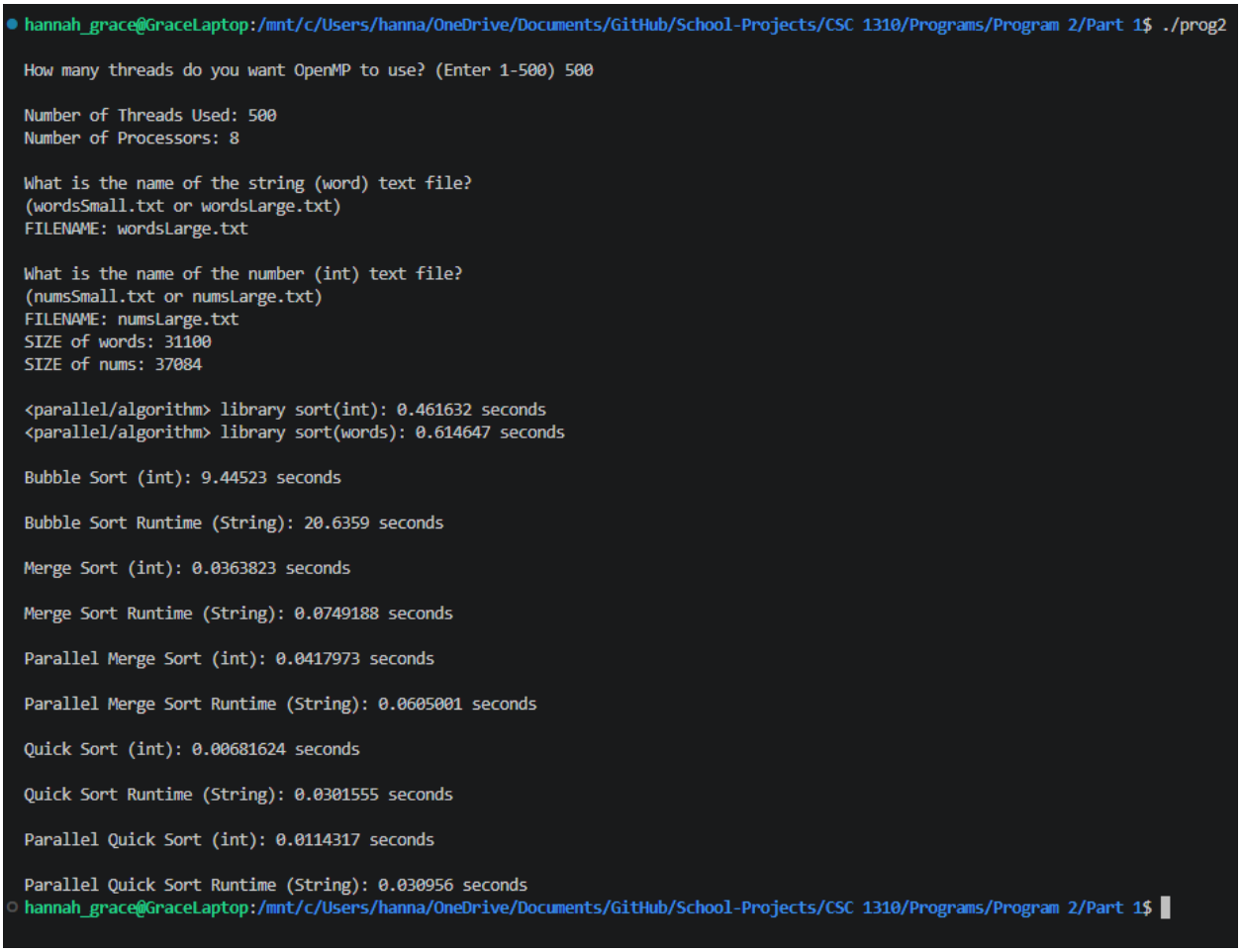
a. Why do you think you got the results that you got?

I believe I got these results through even more threads being used by the processor, and since the data set it large enough it is more efficient.

b. How does it compare to the sample output provided?

It is not as slow as the sample output, but it is still within the range of efficiency

Sample Output THREE:

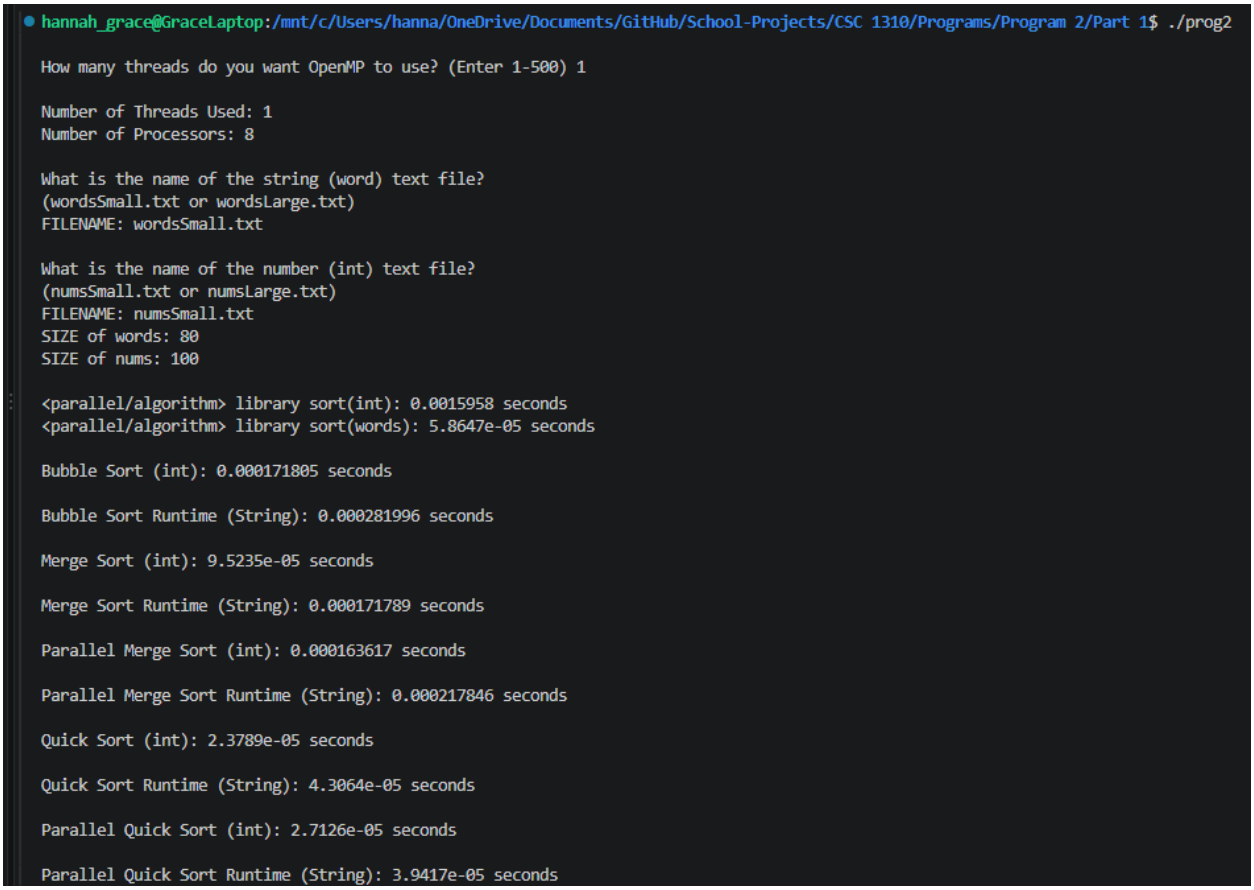


a. Why do you think you got the results that you got?

The results are due to the extra threads being used to sort through the data set at the same time.

b. How does it compare to the sample output provided?
The output here and from the sample is relatively the same with the exception of my bubble sort algorithm being quicker.

Fastest Output:



The fastest thread number was one. I believe this is due to me using the smaller data sets, so allocating extra space for thread usage would be redundant and inefficient, making the single thread faster.